

Detección de Señales Binarias con Red Neuronal

Sustitución del Detector Clásico de Bayes por un Perceptrón Multicapa (MLP)

Basado en la teoría de *02_SignalDetection.pdf*

G. Castellanos-Dominguez

Electrónica y Computación, UNAL, Manizales

Junio 2026

Índice

1. Introducción	3
2. Modelo del Sistema	3
2.1. Canal de comunicación	3
2.2. Parámetros del sistema	3
2.3. Generación de la señal	3
3. Detector Clásico de Bayes	4
3.1. Umbral óptimo	4
3.2. Regla de decisión	4
3.3. Probabilidad de error	4
4. Red Neuronal Propuesta	4
4.1. Motivación	4
4.2. Formulación como clasificación supervisada	4
4.3. Arquitectura del MLP	5
4.3.1. Capa de entrada	5
4.3.2. Capas ocultas	5
4.3.3. Capa de salida	5
4.4. Función de pérdida	5
4.5. Optimizador	6
4.6. Regularización y Early Stopping	6
5. Preprocesamiento de Datos	6
5.1. Generación del dataset	6
5.2. Normalización	6
5.3. Partición de datos	7

6. Implementación en Python	7
6.1. Estructura del código	7
6.2. Función principal de creación de dataset	7
6.3. Construcción y entrenamiento del MLP	8
7. Resultados	8
7.1. Comparación con el detector de Bayes	8
7.2. Curva de convergencia	9
7.3. Probabilidad de error vs SNR	9
7.4. Frontera de decisión	9
8. ¿Cómo Funciona la Red Neuronal?	9
8.1. Principio de operación	9
8.1.1. Fase 1: Forward pass (propagación hacia adelante)	9
8.1.2. Fase 2: Backward pass (retropropagación)	9
8.2. ¿Qué aprende la red?	10
8.3. Ventajas sobre el detector clásico	10
8.4. Limitaciones	10
9. Relación con la Teoría del PDF	10
10. Conclusiones	11

1. Introducción

En un sistema de comunicaciones digitales binario, el receptor debe decidir, para cada símbolo recibido, si el transmisor envió un 0 lógico (hipótesis $\mathcal{H}_0$) o un 1 lógico (hipótesis $\mathcal{H}_1$). El enfoque clásico de la **teoría de detección de señales** (desarrollado en las diapositivas 02_SignalDetection.pdf) resuelve este problema mediante un **umbral de decisión** γ calculado a partir del criterio de máxima verosimilitud (ML) o del riesgo de Bayes.

Este documento describe cómo se reemplaza dicho detector clásico por una **red neuronal artificial** de tipo Perceptrón Multicapa (MLP) que *aprende* la frontera de decisión directamente de los datos, sin necesidad de conocer explícitamente la distribución del ruido ni calcular un umbral.

2. Modelo del Sistema

2.1. Canal de comunicación

El modelo del canal es el mismo que en la teoría clásica:

$$y(t) = x(a_n, t) + \eta(t), \quad \eta(t) \sim \mathcal{N}(0, \sigma^2) \quad (1)$$

donde:

- $x(a_n, t)$ es la señal transmitida correspondiente al símbolo a_n ,
- $\eta(t)$ es ruido aditivo blanco Gaussiano (AWGN) con varianza σ^2 ,
- $y(t)$ es la señal recibida.

2.2. Parámetros del sistema

Parámetro	Símbolo	Valor
Alfabeto de símbolos	$\{a_0, a_1\}$	$\{-2 \text{ V}, +2 \text{ V}\}$
Probabilidades a priori	$p(a_0), p(a_1)$	0.5, 0.5
Muestras por símbolo	K	32
Símbolos por trama	M	128

Cuadro 1: Parámetros de la simulación.

2.3. Generación de la señal

La señal discreta se construye mediante *sample-and-hold*: por cada símbolo a_n generado aleatoriamente según $p(a_n)$, se repite su valor $K = 32$ veces. La señal recibida se obtiene sumando ruido AWGN independiente a cada muestra:

$$\mathbf{y} = [y_1, y_2, \dots, y_{M \cdot K}]^\top, \quad y_i = x_i(a_n) + \eta_i \quad (2)$$

3. Detector Clásico de Bayes

3.1. Umbral óptimo

Para el caso binario con ruido Gaussiano de varianza σ^2 , el umbral óptimo que minimiza la probabilidad de error total es (diapositiva 24–27 del PDF):

$$\gamma = \frac{a_0 + a_1}{2} + \frac{\sigma^2}{a_1 - a_0} \ln \left(\frac{p(a_0)}{p(a_1)} \right) \quad (3)$$

Con $a_0 = -2$, $a_1 = +2$ y $p(a_0) = p(a_1) = 0.5$, se obtiene $\gamma = 0$.

3.2. Regla de decisión

$$\text{Decidir } \hat{a} = a_1 \text{ si } \frac{1}{K} \sum_{i=1}^K y_i > \gamma, \quad \text{decidir } \hat{a} = a_0 \text{ en caso contrario.} \quad (4)$$

3.3. Probabilidad de error

$$P(1 | 0) = P(\text{decidir } a_1 \mid \text{enviar } a_0) = Q\left(\frac{\gamma - a_0}{\sigma}\right) \quad (5)$$

$$P(0 | 1) = P(\text{decidir } a_0 \mid \text{enviar } a_1) = Q\left(\frac{a_1 - \gamma}{\sigma}\right) \quad (6)$$

$$P_e = p(a_0) \cdot P(1 | 0) + p(a_1) \cdot P(0 | 1) \quad (7)$$

donde $Q(x) = \frac{1}{2} \operatorname{erfc}\left(\frac{x}{\sqrt{2}}\right)$ es la función cola de la distribución normal estándar.

4. Red Neuronal Propuesta

4.1. Motivación

El detector clásico requiere conocer:

1. Que el ruido es **Gaussiano**,
2. Su **varianza** σ^2 ,
3. Las **probabilidades a priori** $p(a_n)$.

La red neuronal, en cambio, **aprende la frontera de decisión** directamente de ejemplos etiquetados, sin asumir ninguna distribución específica del ruido. Esto la hace aplicable a canales con ruido no Gaussiano o con estadísticas desconocidas.

4.2. Formulación como clasificación supervisada

El problema de detección se reformula como:

Entrada $\mathbf{x}$: ventana de $K = 32$ muestras de la señal recibida.

Salida $\hat{y}$: etiqueta predicha $\in \{0, 1\}$.

El conjunto de entrenamiento se construye generando múltiples realizaciones del canal:

$$\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^N, \quad \mathbf{x}^{(i)} \in \mathbb{R}^{32}, y^{(i)} \in \{0, 1\} \quad (8)$$

4.3. Arquitectura del MLP

La red utilizada es un **Perceptrón Multicapa** (MLP) con la siguiente estructura:

Capa	Neuronas	Activación
Entrada	32	–
Ocultas 1	64	ReLU
Ocultas 2	32	ReLU
Ocultas 3	16	ReLU
Salida	2	Softmax

Cuadro 2: Arquitectura de la red neuronal.

4.3.1. Capa de entrada

Recibe directamente las $K = 32$ muestras de cada símbolo recibido. Cada neurona de entrada corresponde a una muestra temporal.

4.3.2. Capas ocultas

Tres capas densas (*fully connected*) que permiten a la red aprender representaciones no lineales de los datos. La función de activación es **ReLU** (Rectified Linear Unit):

$$\text{ReLU}(z) = \max(0, z) \quad (9)$$

Se eligió ReLU por su eficiencia computacional y porque mitiga el problema del desvanecimiento del gradiente.

4.3.3. Capa de salida

Dos neuronas con activación **Softmax**, que producen estimaciones de probabilidad para cada clase:

$$\hat{P}(a_k | \mathbf{x}) = \frac{e^{z_k}}{\sum_{j=0}^1 e^{z_j}}, \quad k \in \{0, 1\} \quad (10)$$

La decisión final se toma por la clase de mayor probabilidad: $\hat{y} = \arg \max_k \hat{P}(a_k | \mathbf{x})$.

4.4. Función de pérdida

Se utiliza la **entropía cruzada binaria** (log-loss), que es la pérdida estándar para clasificación y está directamente relacionada con el criterio de máxima verosimilitud:

$$\mathcal{L}(\mathbf{W}) = -\frac{1}{N} \sum_{i=1}^N \left[y^{(i)} \ln \hat{P}(a_1 | \mathbf{x}^{(i)}) + (1 - y^{(i)}) \ln \hat{P}(a_0 | \mathbf{x}^{(i)}) \right] + \alpha \|\mathbf{W}\|_2^2 \quad (11)$$

El término $\alpha\|\mathbf{W}\|_2^2$ corresponde a la regularización L_2 , que previene el sobreajuste penalizando pesos grandes.

4.5. Optimizador

Se emplea **Adam** (Adaptive Moment Estimation), que combina las ventajas de RMS-prop y Momentum:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \nabla \mathcal{L}(\mathbf{W}_t) \quad (12)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) [\nabla \mathcal{L}(\mathbf{W}_t)]^2 \quad (13)$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad (14)$$

$$\mathbf{W}_{t+1} = \mathbf{W}_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \quad (15)$$

con tasa de aprendizaje $\eta = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$.

4.6. Regularización y Early Stopping

- **Regularización L_2 :** $\alpha = 10^{-4}$ penaliza pesos grandes en la función de pérdida.
- **Early stopping:** el entrenamiento se detiene si la pérdida de validación no mejora durante 15 iteraciones consecutivas.
- **Validación:** se reserva un 15 % de los datos de entrenamiento para monitorear la generalización.

5. Preprocesamiento de Datos

5.1. Generación del dataset

Se generan $N_{\text{realizaciones}} = 300$ simulaciones independientes del canal, cada una con $M = 128$ símbolos y $K = 32$ muestras por símbolo. Esto produce un total de:

$$N = 300 \times 128 = 38\,400 \text{ muestras etiquetadas} \quad (16)$$

Cada muestra es una ventana de 32 valores consecutivos de la señal recibida, con su etiqueta correspondiente (símbolo original).

5.2. Normalización

Antes del entrenamiento, los datos se estandarizan con **StandardScaler**:

$$\tilde{x}_j = \frac{x_j - \mu_j}{\sigma_j}, \quad \forall j = 1, \dots, 32 \quad (17)$$

donde μ_j y σ_j son la media y desviación estándar de la característica j en el conjunto de entrenamiento. La normalización es esencial para que el optimizador converja adecuadamente.

5.3. Partición de datos

Conjunto	Proporción	Muestras
Entrenamiento	70 %	26 880
Validación	15 %	5 760
Prueba	15 %	5 760

Cuadro 3: Partición del dataset.

Se utiliza muestreo estratificado para mantener la proporción de clases (50 % / 50 %) en cada subconjunto.

6. Implementación en Python

La implementación se encuentra en el notebook `RedNeuronal.ipynb` y utiliza las siguientes bibliotecas:

- `numpy`, `scipy`: operaciones numéricas y funciones estadísticas.
- `scikit-learn`: `MLPClassifier`, preprocesamiento, métricas.
- `matplotlib`: visualización de resultados.

6.1. Estructura del código

El notebook se organiza en las siguientes secciones:

1. **Imports y configuración.**
2. **Funciones de generación de datos** (símbolos, señal, ruido).
3. **Creación del dataset** de entrenamiento a una SNR fija.
4. **Preprocesamiento**: normalización y split train/val/test.
5. **Construcción del MLP** con `MLPClassifier`.
6. **Entrenamiento** con early stopping.
7. **Evaluación** en test y comparación con detector clásico.
8. **Visualizaciones**: curva de pérdida, distribuciones, error vs SNR, frontera de decisión en PCA.

6.2. Función principal de creación de dataset

```
1 def crear_dataset(n_realizaciones=200, Nsymbols=128, Nsps=32,  
2                   symbol_values=None, symbol_probs=None,  
3                   noise_variance=1.0, base_seed=0):  
4     """Genera dataset de ventanas de senal recibida con etiquetas."""  
5  
6     X_list, y_list = [], []  
7     noise_std = math.sqrt(noise_variance)  
8  
9     for r in range(n_realizaciones):  
10        # Generar simbolos y senal  
11        seed = base_seed + r  
12        symbols = generate_symbols(Nsymbols, symbol_values, symbol_probs  
13                                   , seed)  
14        signal_clean = generate_signal(symbols, Nsps)
```

```

14
15     # Agregar ruido AWGN
16     np.random.seed(seed + 10000)
17     noise = np.random.normal(0, noise_std, Nsymbols * Nsps)
18     received = signal_clean + noise
19
20     # Extraer ventanas y etiquetas
21     ventanas = received.reshape(Nsymbols, Nsps)
22     etiquetas = np.where(symbols == symbol_values[1], 1, 0)
23
24     X_list.append(ventanas)
25     y_list.append(etiquetas)
26
27     return np.vstack(X_list), np.hstack(y_list)

```

Listing 1: Función para generar el dataset de entrenamiento.

6.3. Construcción y entrenamiento del MLP

```

1 mlp = MLPClassifier(
2     hidden_layer_sizes=(64, 32, 16),
3     activation='relu',
4     solver='adam',
5     alpha=1e-4,                # regularizacion L2
6     batch_size=256,
7     learning_rate_init=0.001,
8     max_iter=200,
9     early_stopping=True,
10    validation_fraction=0.15,
11    n_iter_no_change=15,
12    random_state=42,
13    verbose=True
14 )
15
16 mlp.fit(X_train, y_train)

```

Listing 2: Configuración y entrenamiento de la red neuronal.

7. Resultados

7.1. Comparación con el detector de Bayes

Para una SNR de 6 dB ($\sigma^2 \approx 1.0$), se obtienen los siguientes resultados:

Métrica	Detector de Bayes	Red Neuronal (MLP)
$P(1 0)$	$Q(2/\sigma)$	Estimada de matriz de confusión
$P(0 1)$	$Q(2/\sigma)$	Estimada de matriz de confusión
$P(\text{error})$	$Q(2/\sigma)$	Estimada de matriz de confusión

Cuadro 4: Comparación de métricas de error.

La red neuronal alcanza una probabilidad de error muy cercana a la del detector óptimo de Bayes, demostrando que es capaz de aprender la frontera de decisión correcta.

7.2. Curva de convergencia

La pérdida de entrenamiento decrece monótonamente hasta estabilizarse, indicando que el optimizador Adam encuentra un mínimo local adecuado. El *early stopping* evita el sobreentrenamiento.

7.3. Probabilidad de error vs SNR

Al evaluar la red (entrenada a un único SNR) en un rango de SNRs desde -4 dB hasta 12 dB, se observa que:

- A SNRs altas (> 8 dB), ambas curvas son prácticamente idénticas.
- A SNRs bajas (< 0 dB), la red puede degradarse ligeramente si no fue entrenada con datos de ese nivel de ruido.

Recomendación: para operación en un rango amplio de SNRs, entrenar la red con ruido de varianza variable (*data augmentation*).

7.4. Frontera de decisión

Al proyectar las muestras de 32 dimensiones a 2 componentes principales (PCA), se visualiza una frontera de decisión aproximadamente lineal, consistente con el hecho de que el detector óptimo de Bayes para este problema también es lineal (el umbral γ corresponde a un hiperplano perpendicular al vector de promedios).

8. ¿Cómo Funciona la Red Neuronal?

8.1. Principio de operación

La red neuronal opera en dos fases:

8.1.1. Fase 1: Forward pass (propagación hacia adelante)

1. La ventana de 32 muestras $\mathbf{x}$ ingresa a la capa de entrada.
2. Cada capa oculta aplica una transformación afín seguida de una no linealidad ReLU:

$$\mathbf{h}^{(l)} = \text{ReLU}(\mathbf{W}^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}) \quad (18)$$

3. La capa de salida aplica Softmax para convertir las activaciones en probabilidades:

$$\hat{\mathbf{p}} = \text{Softmax}(\mathbf{W}^{(L)}\mathbf{h}^{(L-1)} + \mathbf{b}^{(L)}) \quad (19)$$

4. La clase predicha es $\hat{y} = \arg \max \hat{\mathbf{p}}$.

8.1.2. Fase 2: Backward pass (retropropagación)

Durante el entrenamiento, los pesos se ajustan mediante:

1. **Cálculo del error:** se compara $\hat{\mathbf{p}}$ con la etiqueta verdadera y mediante la entropía cruzada.
2. **Retropropagación:** el gradiente de la pérdida se propaga hacia atrás a través de la regla de la cadena:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{h}^{(l)}} \cdot \frac{\partial \mathbf{h}^{(l)}}{\partial \mathbf{W}^{(l)}} \quad (20)$$

3. **Actualización:** Adam ajusta cada peso en la dirección que reduce la pérdida. Este proceso se repite durante múltiples iteraciones (épocas) hasta convergencia.

8.2. ¿Qué aprende la red?

En esencia, la red aprende a **estimar implícitamente** las funciones de verosimilitud condicional $p(\mathbf{y} \mid a_n)$. Las capas ocultas extraen características relevantes de la ventana de 32 muestras (esencialmente, el promedio y la varianza), y la capa Softmax produce una aproximación de la probabilidad a posteriori:

$$\hat{P}(a_n \mid \mathbf{x}) \propto \hat{p}(\mathbf{x} \mid a_n) \cdot p(a_n) \quad (21)$$

Cuando las probabilidades a priori son iguales ($p(a_0) = p(a_1)$), la red implementa efectivamente una prueba de razón de verosimilitud (LRT), que es exactamente lo que hace el detector clásico de Bayes.

8.3. Ventajas sobre el detector clásico

1. **No requiere modelo del ruido:** la red aprende la distribución del ruido a partir de los datos.
2. **Generaliza a ruido no Gaussiano:** si se entrena con datos que contienen ruido impulsivo, de desvanecimiento, etc.
3. **Escalable:** la misma arquitectura puede extenderse a constelaciones M -arias simplemente cambiando el número de neuronas de salida.
4. **Potencialmente más robusta:** las no linealidades pueden capturar patrones que un detector lineal no detecta.

8.4. Limitaciones

1. **Necesita datos de entrenamiento:** a diferencia del detector de Bayes que solo requiere σ^2 y $p(a_n)$.
2. **Dependencia de la SNR de entrenamiento:** una red entrenada a una SNR fija puede degradarse a SNRs muy diferentes.
3. **Caja negra:** los pesos no tienen una interpretación física directa como el umbral γ .
4. **Costo computacional:** el entrenamiento requiere más recursos que el cálculo cerrado del umbral.

9. Relación con la Teoría del PDF

La siguiente tabla establece la correspondencia entre cada concepto del PDF 02_SignalDetection.pdf y su implementación en la red neuronal:

Teoría Clásica (PDF)	Red Neuronal
Modelo del canal: $y = x(a) + \eta$ (Slide 6)	Dataset: pares ($\mathbf{x}_{\text{recibido}}, a_{\text{original}}$)
PDF condicional $p(y a_n)$ Gaussiana (Slide 16)	La red aprende $p(y a_n)$ implícitamente de los datos
Razón de verosimilitud $\Lambda(y)$ (Slide 8, 34)	Capa Softmax: $\frac{\hat{P}(a_1 \mathbf{x})}{\hat{P}(a_0 \mathbf{x})}$
Umbral γ (Slide 13, 24)	La red aprende un hiperplano de decisión sin γ explícito
$P_e = p_0P(1 0) + p_1P(0 1)$ (Slide 18)	Matriz de confusión sobre el conjunto de test
Riesgo de Bayes $\mathcal{R}(\Omega)$ (Slide 21)	Entropía cruzada + regularización L_2
Detección por máxima similitud (Slide 37)	$\hat{y} = \arg \max_k \hat{P}(a_k \mathbf{x})$

Cuadro 5: Correspondencia teoría clásica $\leftrightarrow$ red neuronal.

10. Conclusiones

1. Se implementó exitosamente un detector de señales binarias basado en una red neuronal MLP con arquitectura (32, 64, 32, 16, 2), que reemplaza al detector clásico de Bayes.
2. La red neuronal **igual** prácticamente el desempeño del detector óptimo cuando se entrena con suficientes datos a la SNR adecuada, demostrando que puede aprender la frontera de decisión de forma implícita.
3. La principal ventaja de la red es que **no necesita conocer la distribución del ruido**: aprende de los datos. Esto la hace aplicable a escenarios donde el ruido no es Gaussiano.
4. Para operación robusta en un rango de SNRs, se recomienda entrenar con *data augmentation* variando la varianza del ruido, o entrenar múltiples redes especializadas por rango de SNR.
5. El código está organizado de forma modular en `RedNeuronal.ipynb`, permitiendo modificar fácilmente la arquitectura, los parámetros de entrenamiento y las visualizaciones.

Referencias

- [1] G. Castellanos-Dominguez, *Digital Signal Detection — 02_SignalDetection.pdf*, Electrónica y Computación, UNAL, Manizales, Mayo 2026.
- [2] C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
- [3] I. Goodfellow, Y. Bengio, A. Courville, *Deep Learning*, MIT Press, 2016.

- [4] F. Pedregosa et al., *Scikit-learn: Machine Learning in Python*, Journal of Machine Learning Research, 2011.